

Machine Learning for Modelling: *Supervised Learning*

1

Introduction

- We have already introduced CNNs:

 1. Specialized for processing data that lie on a regular grid.
 2. Particularly suited to processing images, which have a very large number of input variables (precluding the use of fully connected networks)
 3. Behave similarly at every position (parameter sharing, remember?).

2

Introduction

- Today we introduce transformers.
- They were initially targeted at natural language processing (NLP) problems, where the network input is a series of high-dimensional embeddings representing words or word fragments.
- Language datasets share some of the characteristics of image data:
 1. The number of input variables can be very large.
 2. The statistics are similar at every position (no need to re-learn the meaning of the word *dog* at every possible position in a body of text).
 3. Language datasets have the complication that input sequences are of variable length, and unlike images, there is no easy way to resize them.

3

Transformers

4

Processing text data

- Consider the following passage:
"The restaurant refused to serve me a ham sandwich because it only cooks vegetarian food. In the end, they just gave me two slices of bread. Their ambiance was just as good as the food and service."
- The goal is to design a NN to process this text into a representation suitable for downstream tasks (e.g., it might be used to classify the review as positive or negative, or to answer questions such as "Does the restaurant serve steak?")

5

Processing text data

We can make some observations:

1. The encoded input can be very large. Assuming a 1024-d embedding, 37 words lead to an input length of $37 \times 1024 = 37888$. A more realistic body of text might have 100s/1000s of words $\rightarrow$ fully connected NN are impractical
2. One characteristic of NLP problems is that each input is of a different length $\rightarrow$ not even obvious how to apply a fully connected NN
 - this suggests that the NN should share parameters across words at different input positions (similarly to how CNNs share parameters across different image positions)
3. Language is fundamentally ambiguous. It is unclear from the syntax alone that the pronoun **it** refers to the restaurant and not to the ham sandwich. To understand the text, the word **it** should be connected to the word **restaurant**.

6

Processing text data

We can make some observations:

1. The encoded input can be very large. Assuming a 1024-d embedding, 37 words lead to an input length of $37 \times 1024 = 37888$. A more realistic body of text might have 100s/1000s of words
2. On the other hand, the input is of a different length for each word. For example, the word "The" has a 1024-d embedding, while the word "a" has a 1024-d embedding. This is problematic for fully connected NNs.
3. Language is fundamentally ambiguous. It is unclear from the syntax alone that the pronoun **it** refers to the restaurant and not to the ham sandwich. To understand the text, the word **it** should be connected to the word **restaurant**.

7

Transformers

Dot-product self-attention

8

Dot-product self-attention

Therefore, a model for processing text will:

1. Use parameter sharing to cope with long input passages of differing lengths
2. Contain connections between word representations that depend on the words themselves

The transformer acquires both properties by using dot-product self-attention.

9

Dot-product self-attention

A standard NN layer $f[x]$ takes a $D \times 1$ input x and applies a linear transformation followed by a non-linear activation function $a[\cdot]$:

$$f[x] = a[\beta + \Omega x]$$

where β contains the biases and Ω contains the weights.

A self-attention block $sa[\cdot]$ takes N inputs x_n , each of dimension $D \times 1$, and returns N output vectors of the same size.

In the context of NLP, each of the inputs x_n will represent a word or a word fragment.

10

Dot-product self-attention

First, a set of **values** are computed for each input:

$$v_n = \beta_v + \Omega_v x_n$$

where β_v contains the biases and Ω_v contains the weights.

Then the n -th output $sa[x_n]$ is a weighted sum of all the values v_n :

$$sa[x_n] = \sum_{m=1}^N a[x_m, x_n] v_m$$

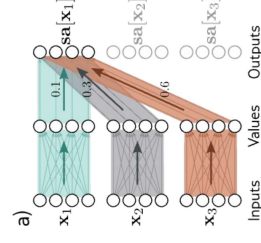
The scalar weight $a[x_m, x_n]$ is the **attention** that input x_n pays to input x_m .

The N weights $a[\cdot, x_n]$ are non-negative and sum to one.

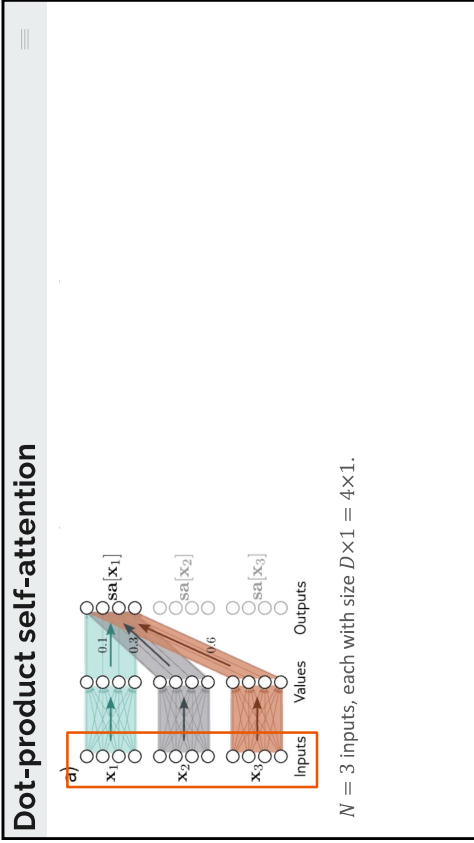
Self-attention can be thought of as **routing** the values in different proportions to create each output.

11

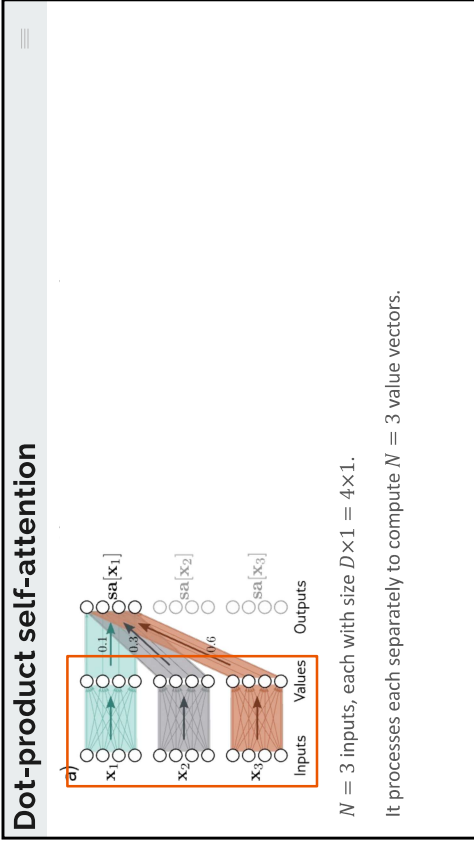
Dot-product self-attention



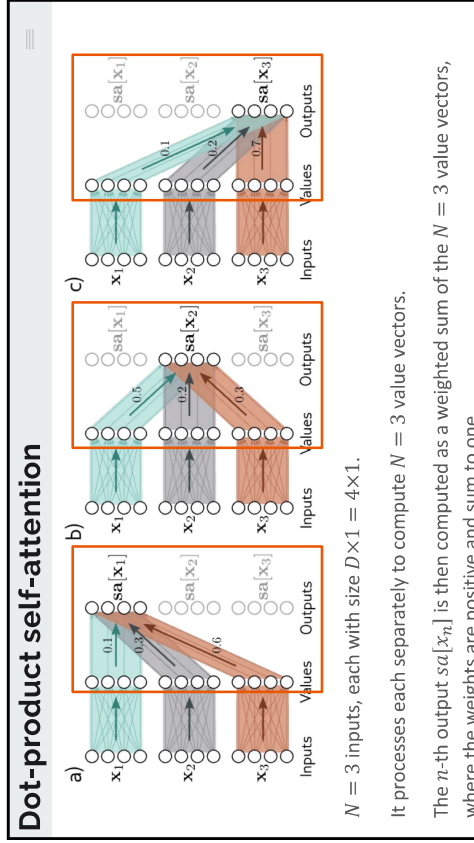
12



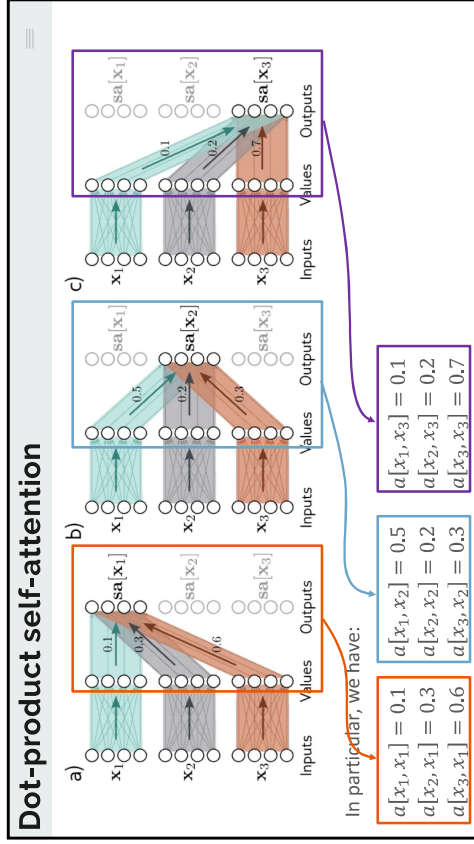
13



14



15



16

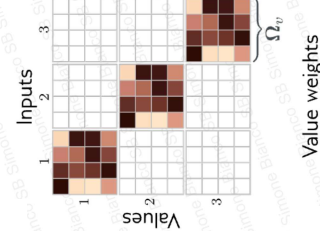
Computing and weighting values

To compute the values, the same weights Ω_v (with size $D \times D$) and biases β_v (with size $D \times 1$) are applied to each input x_n (with size $D \times 1$):

$$v_n = \beta_v + \Omega_v x_n$$

This computation scales linearly with the sequence length N , so it requires less parameters than a FC NN connecting all DN inputs to the DN outputs.

The value computation can be thus viewed as a sparse matrix multiplication with shared parameters.



Computing and weighting values

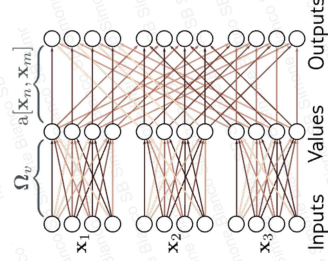
To compute the values, the same weights Ω_v (with size $D \times D$) and biases β_v (with size $D \times 1$) are applied to each input x_n (with size $D \times 1$):

$$v_n = \beta_v + \Omega_v x_n$$

This computation scales linearly with the sequence length N so it requires less parameters than a FC NN connecting all DN inputs to the DN outputs.

The value computation can be thus viewed as a sparse matrix multiplication with shared parameters.

The attention weights $a[x_m, x_n]$ combine the values from different inputs. They are also sparse, since there is only one weight for each ordered pair of inputs (x_m, x_n) regardless of the size of these inputs.



Computing and weighting values

To compute the values, the same weights Ω_v ($D \times D$) and biases β_v ($D \times 1$) are applied to each input x_n ($D \times 1$):

$$v_n = \beta_v + \Omega_v x_n$$

This computation scales linearly with the sequence length N so it requires less parameters than a FC NN connecting all DN inputs to the DN outputs.

The value computation can be thus viewed as a sparse matrix multiplication with shared parameters.

The attention weights $a[x_m, x_n]$ combine the values from different inputs. They are also sparse, since there is only one weight for each ordered pair of inputs (x_m, x_n) regardless of the size of these inputs.

The number of attention weights has a quadratic dependence on the sequence length N but it is independent of the length D of each input x_n .

22

Computing attention weights

We have seen that the outputs result from **two chained linear transformations**:

- The value vectors are computed independently from each input,
- and these vectors are linearly combined by the attention weights.

However, the overall self-attention is **nonlinear** because the attention weights are themselves nonlinear functions of the input.

This is an example of a **hypernetwork**, where one network branch computes the weights of another branch.

24

Computing attention weights

To compute the attention, we apply two more linear transformations to the inputs:

$$q_n = \beta_q + \Omega_q x_n$$

$$k_n = \beta_k + \Omega_k x_n$$

where q_n and k_n are referred to as the **queries** and the **keys**.

We then compute dot products between the queries and the keys followed by a softmax:

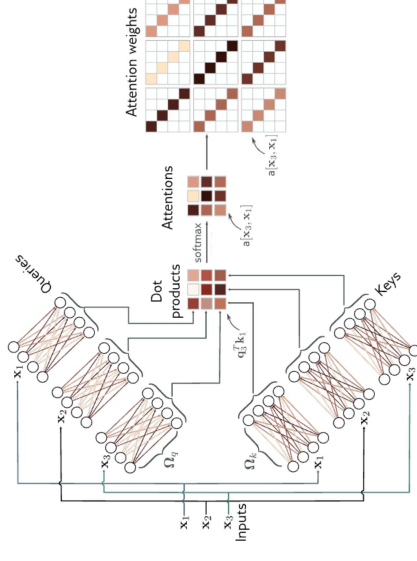
$$a[x_m, x_n] = \text{softmax}_m[k_m^T q_n] = \frac{\exp[k_m^T q_n]}{\sum_{m'=1}^N \exp[k_{m'}^T q_n]}$$

so, for each x_n they are positive and sum to one.

For obvious reasons, this is known as **dot-product self-attention**.

25

Computing attention weights



26

Computing attention weights

The names “queries” and “keys” were inherited from the field of information retrieval and have the following interpretation:

- the dot product operation returns a measure of similarity between its inputs, so the weights $a[x_*, x_n]$ depend on the relative similarities between each query and the keys.
- The softmax function means that we can think of the key vectors as “competing” with one another to contribute to the final result.

The queries and keys must have the same dimensions.

However, these can differ from the dimension of the values, which is usually the same size as the input so that the representation does not change size.

27

Self-attention summary

The n -th output is a weighted sum of the same linear transformation $v_* = \beta_v + \Omega_v x_*$ applied to all the inputs, where these attention weights are positive and sum to one.

The weights depend on a measure of similarity between input x_n and the other inputs.

There is no activation function, but the mechanism is nonlinear due to the dot-product and a softmax operation used to compute the attention weights.

This mechanism fulfills the initial requirements:

1. There is a single shared set of parameters $\phi = \{\beta_v, \Omega_v, \beta_q, \Omega_q, \beta_k, \Omega_k\}$. This is independent of the number of inputs N , so the networks can be applied to difference sequence lengths.
2. There are connections between the inputs (words), and the strength of these connections depends on the inputs themselves via the attention weights.

28

Matrix form

The previous computation can be written in a compact form if the N inputs x_n form the columns of the $D \times N$ matrix X .

The values, queries, and keys can be computed as:

$$V[X] = \beta_v 1^T + \Omega_v X$$

$$Q[X] = \beta_q 1^T + \Omega_q X$$

$$K[X] = \beta_k 1^T + \Omega_k X$$

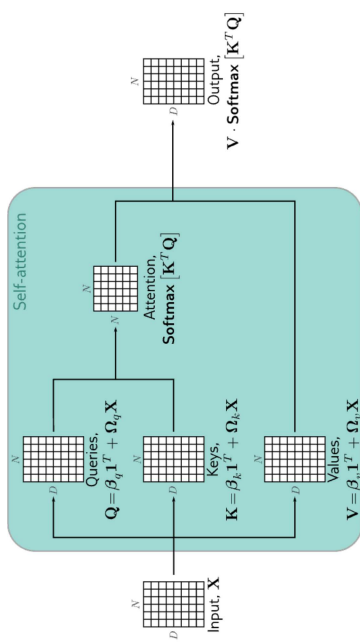
The self-attention computation is then:

$$Sa[X] = V[X] \cdot \text{Softmax}[K[X]^T Q[X]] = V \cdot \text{Softmax}[K^T Q]$$

where softmax is independently applied on the columns of its input.

29

Matrix form



30

Transformers

Extensions to dot-product self-attention

Positional encoding

The self-attention mechanism discards important information: the computation is the same regardless of the order of the inputs.

However, the order is important when the inputs correspond to the words in a sentence. E.g., the sentence *The woman ate the raccoon* has a different meaning than *The raccoon ate the woman*.

There are two main approaches to incorporate position information:

- Absolute position embeddings
- Relative position embeddings

Positional encoding: absolute position embeddings

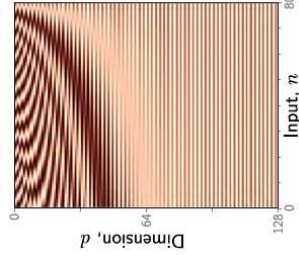
A matrix Π is added to the input X that encodes positional information.

Each column of Π is unique and contains information about the absolute position in the input sequence.

This matrix can be chosen by hand (e.g., sinusoidal pattern) or learned.

It may be added to the network inputs or at every network layer.

Sometimes it is added to X in the computation of the queries and keys but not to the values.



Positional encoding: relative position embeddings

The input to a self-attention mechanism may be an entire sentence, many sentences, or just a fragment of a sentence, and the absolute position of a word is much less important than the relative position between two inputs.

Of course, this can be recovered if the system knows the absolute position of both, but relative position embeddings encode this information directly.

Each element of the attention matrix corresponds to a particular offset between query position a and key position b .

Relative position embeddings learn a parameter $\pi_{a,b}$ for each offset and use this to modify the attention matrix by adding these values, multiplying by them, or using them to alter the attention matrix in some other way.

Scaled dot product self-attention

The dot products in the attention computation can have large magnitudes and move the arguments to the softmax function into a region where the largest value completely dominates.

Small changes to the inputs to the softmax function now have little effect on the output (i.e., the gradients are very small), making the model difficult to train.

To prevent this, the dot products are scaled by the square root of the dimension D_q of the queries and keys (i.e., the number of rows in Ω_q and Ω_k , which must be the same):

$$Sa[X] = V \cdot Softmax \left[\frac{K^T Q}{\sqrt{D_q}} \right]$$

35

Multiple heads

Multiple self-attention mechanisms are usually applied in parallel, and this is known as multi-head self-attention. Now H different sets of values, keys and queries are computed:

$$V_h[X] = \beta_{vh} 1^T + \Omega_{vh} X$$

$$Q_h[X] = \beta_{qh} 1^T + \Omega_{qh} X$$

$$K_h[X] = \beta_{kh} 1^T + \Omega_{kh} X$$

The h -th self-attention mechanism or **head** can be written as:

$$Sa_h[X] = V_h \cdot Softmax \left[\frac{K_h^T Q_h}{\sqrt{D_q}} \right]$$

where we have different parameters for each head.

Typically, if the dimension of the inputs is D and there are H heads, the values, queries, and keys will all be of size D/H as this allows for an efficient implementation.

37

Multiple heads

The outputs of these self-attention mechanisms are vertically concatenated, and another linear transform Ω_c is applied to combine them:

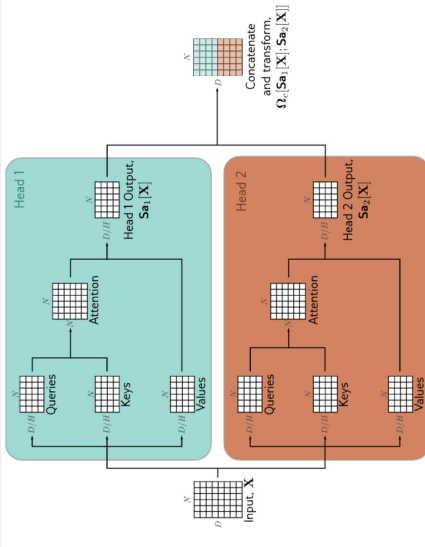
$$MhSa[X] = \Omega_c [Sa_1[X]^T, Sa_2[X]^T, \dots, Sa_H[X]^T]$$

Multiple heads seem to be necessary to make the transformer work well.

It has been speculated that they make the self-attention network more robust to bad initializations.

38

Multiple heads



39

Transformers

Transformer layers

Transformer layers

Self-attention is just one part of a larger transformer layer.

This consists of a multi-head self-attention unit (which allows the word representations to interact with each other) followed by a fully connected network $mhp[x_+]$ (that operates separately on each word).

Both units are residual networks (i.e., their output is added back to the original input).

In addition, it is typical to add a *LayerNorm* operation after both the self-attention and fully connected networks (it is similar to BatchNorm, but uses statistics across the tokens within a single input sequence to perform the normalization).

In a real network, the data passes through a series of these layers.

Transformer layers

